# Week 3 Day-by-Day Plan: Docker + Streamlit Dashboard + Polish

## Containerising Your API and Building an Interactive Frontend

> **Goal**: By end of Week 3, your entire project runs with one command (`docker-compose up`), has an interactive Streamlit dashboard for live demos, and is portfolio-ready with a polished GitHub repo.

---

## What You Already Have (from Weeks 1-2)

| Component | Status |
|---|---|
| ML model (CitizenLab DistilBERT) | ✅ Working, 91.7% accuracy |
| FastAPI API (5 endpoints) | ✅ Working, all tested |
| Language detection + aliases | ✅ Working |
| Input sanitisation + edge cases | ✅ Working |
| 31 pytest tests | ✅ All passing |
| Performance benchmarks | ✅ Documented |
| README + API_DOCS | ✅ Written |
| Dockerfile | ⚠️ Exists but needs Python version fix |
| docker-compose.yml | ⚠️ Exists, minor tweaks needed |
| streamlit_app.py | 🔴 Placeholder only |

---

## Pre-Setup: Docker Installation (15 min)

If you don't have Docker installed:

1. Download **Docker Desktop** from https://www.docker.com/products/docker-desktop/
2. Install and restart your computer
3. Open Docker Desktop — wait for it to say "Docker is running"
4. Verify in PowerShell:

```powershell
```

```
docker --version
docker-compose --version
```

If Docker is already installed, skip to Day 1.

---

## Day 1 (Monday) — Docker Fundamentals + Fix Dockerfile (2-3 hours)

### What to learn

1. **What is Docker?** — It packages your app + all dependencies into a single "container" that runs identically everywhere. No more "works on my machine" problems. Think of it like a shipping container — everything inside is self-contained.
2. **What is a Dockerfile?** — A recipe that tells Docker how to build your container. It lists the base image (Python 3.12), copies your code, installs dependencies, and sets the startup command.
3. **What is a multi-stage build?** — Your Dockerfile has two stages:
   - Stage 1 (builder): Install all Python packages
   - Stage 2 (runtime): Copy only the installed packages + your code This keeps the final image smaller because build tools aren't included.
4. **What is docker-compose?** — Runs multiple containers together. Your project has two services: the API (FastAPI) and the dashboard (Streamlit). docker-compose connects them so Streamlit can call the API internally.

### What to do

**Step 1: Fix the Dockerfile** Your current Dockerfile uses Python 3.10 but you're developing with Python 3.12. Update the base image and add a few improvements:

- Change `python:3.10-slim` to `python:3.12-slim`
- Add `.dockerignore` to skip unnecessary files (venv, **pycache**, .git)
- The HEALTHCHECK uses `curl` but `python:3.12-slim` doesn't have curl — fix this

**Step 2: Create `.dockerignore`** This prevents copying unnecessary files into the Docker image (like your venv, .git folder, **pycache**, etc.)

**Step 3: Build and run the API container**

```powershell
docker build -t content-moderator .
docker run -p 8000:8000 content-moderator
```

Then test: `http://localhost:8000/docs`

**Step 4: Understand what happened** Docker downloaded Python 3.12, installed all your pip packages, copied your code, and started uvicorn — all inside an isolated container. Your local venv wasn't used at all.

### Done when

- [ ] Dockerfile updated with Python 3.12
- [ ] `.dockerignore` created
- [ ] `docker build` succeeds
- [ ] `docker run` starts the API
- [ ] Can access Swagger docs from the container at `http://localhost:8000/docs`
- [ ] `git commit -m "Day 1: Dockerfile fixed, containerised API working"`

---

## Day 2 (Tuesday) — Docker Compose + Both Services (2-3 hours)

### What to learn

1. **docker-compose** — Runs multiple containers from one command
2. **Service networking** — The API and Streamlit containers can talk to each other using service names (e.g., `http://api:8000` inside the Streamlit container)
3. **Volume mounts** — Persist the Hugging Face model cache so it doesn't re-download every time you restart

### What to do

**Step 1: Update docker-compose.yml**

- Fix Python version consistency
- Add environment variables for API URL in Streamlit service
- Add model cache volume

**Step 2: Build and run both services**

```powershell
docker-compose up --build
```

This should start both the API (port 8000) and Streamlit (port 8501).

**Step 3: Test both services**

- API: `http://localhost:8000/docs`

- Streamlit: `http://localhost:8501` (will show placeholder for now)

### Step 4: Useful Docker commands to know

```powershell
docker-compose up --build       # Build and start
docker-compose down             # Stop everything
docker-compose logs api         # View API logs
docker-compose logs streamlit   # View Streamlit logs
docker ps                       # See running containers
docker images                   # See built images
```

### Done when

- [ ] `docker-compose up --build` starts both services
- [ ] API accessible at `http://localhost:8000/docs`
- [ ] Streamlit accessible at `http://localhost:8501`
- [ ] Model cache persists between restarts (second startup is faster)
- [ ] `git commit -m "Day 2: docker-compose running API + Streamlit"`

---

## Day 3 (Wednesday) — Streamlit Dashboard Part 1: Single Text Moderation (2-3 hours)

### What to learn

1. **What is Streamlit?** — A Python framework for building interactive web apps with pure Python. No HTML/CSS/JavaScript needed. You write Python, Streamlit renders it as a web page with inputs, buttons, charts, etc.
2. **How Streamlit works** — Every time the user interacts (clicks a button, moves a slider), Streamlit re-runs your entire script top to bottom. State is managed with `st.session_state`.
3. **Streamlit + your API** — The dashboard calls your FastAPI endpoints using the `requests` library, then displays the results with Streamlit components.

### What to do

### Step 1: Build the single-text moderation section

- Text input area (st.text_area)
- Threshold slider (st.slider)
- "Analyse" button (st.button)
- Call `POST /api/v1/moderate` with the text

- Display verdict with colour coding (red for toxic, green for clean)
- Show category scores as a bar chart
- Show language detected, threshold used, processing time

**Step 2: Add example texts**

- Preset buttons for common test texts in English, Hindi, Arabic
- Users can click to auto-fill the text area

**Step 3: Test locally (without Docker first)**

```powershell
# Terminal 1: Run API
uvicorn app.main:app --reload --port 8000

# Terminal 2: Run Streamlit
streamlit run streamlit_app.py
```

**Done when**

- [ ] Text input + Analyse button works
- [ ] Verdict displayed with colour (green/red)
- [ ] Category scores shown as bar chart
- [ ] Language, threshold, processing time displayed
- [ ] Example text buttons work
- [ ] `git commit -m "Day 3: Streamlit single-text moderation working"`

---

## Day 4 (Thursday) — Streamlit Dashboard Part 2: Batch Upload + Charts (2-3 hours)

**What to learn**

1. **File upload —** `st.file_uploader` lets users upload CSV files
2. **DataFrames in Streamlit —** `st.dataframe` renders pandas DataFrames as interactive tables
3. **Plotly charts —** Interactive charts (pie charts, bar charts) that look professional in the dashboard

**What to do**

**Step 1: Add CSV batch upload section**

- File uploader for CSV (expects a "text" column)

- Process each row through `POST /api/v1/moderate/batch`
- Display results as a table with colour-coded verdicts
- Show summary stats: total, flagged count, clean count

**Step 2: Add visualisation charts**

- Pie chart: toxic vs clean distribution
- Bar chart: language distribution of uploaded texts
- Bar chart: score distribution (histogram)

**Step 3: Add a results download button**

- Let users download the moderation results as CSV
- Include all fields: text, language, verdict, score, threshold_used

**Done when**

- [ ] CSV upload works
- [ ] Results displayed as interactive table
- [ ] Pie chart shows toxic/clean split
- [ ] Language distribution chart works
- [ ] Download results as CSV works
- [ ] `git commit -m "Day 4: Batch upload, charts, CSV download"`

---

## Day 5 (Friday) — Streamlit Dashboard Part 3: Polish + History (2-3 hours)

**What to do**

**Step 1: Add moderation history**

- Store results in `st.session_state` so they persist during the session
- Display history as a table below the main input
- Add a "Clear history" button

**Step 2: Add sidebar with settings**

- Model info (name, device)
- API health status indicator
- Supported languages list
- Link to API docs

**Step 3: Visual polish**

- Clean layout with columns (st.columns)
- Consistent colour scheme
- Loading spinners while API calls are in progress (st.spinner)
- Error handling for API connection failures

**Step 4: Test the complete dashboard**

- Single text in all 3 languages
- CSV upload with mixed texts
- Edge cases (short text, whitespace, emojis)
- Verify charts render correctly

**Done when**

- [ ] History section works
- [ ] Sidebar shows model info and health
- [ ] Loading spinners during API calls
- [ ] Error handling for API down scenarios
- [ ] Dashboard looks professional
- [ ] `git commit -m "Day 5: Dashboard polished with history, sidebar, error handling"`

---

## Day 6 (Saturday) — Full Integration Testing + Demo Recording (2-3 hours)

**What to do**

**Step 1: Test everything with Docker Compose**

```powershell
docker-compose up --build
```

Test the full flow:

- Open Streamlit at `http://localhost:8501`
- Moderate texts in all 3 languages
- Upload a CSV
- Check that charts render
- Verify API docs still work at `http://localhost:8000/docs`

**Step 2: Record a demo GIF** Use a screen recording tool (e.g., ScreenToGif for Windows) to record:

- Typing toxic text → seeing red "TOXIC" verdict
- Typing clean text → seeing green "CLEAN" verdict
- Switching languages (Hindi, Arabic)
- Uploading a CSV → seeing batch results Save as `docs/demo.gif` and add to README

**Step 3: Final test suite run**

```powershell
pytest tests/test_moderator.py -v
python tests/test_performance.py
```

Make sure everything still passes after all changes.

**Done when**

- [ ] `docker-compose up` runs everything perfectly
- [ ] Demo GIF recorded and added to README
- [ ] All 31 tests still pass
- [ ] Performance benchmark still shows 91.7% accuracy
- [ ] `git commit -m "Day 6: Full integration tested, demo GIF recorded"`

---

## Day 7 (Sunday) — Final Documentation + Portfolio Polish (1-2 hours)

**What to do**

**Step 1: Final README update**

- Add demo GIF: `![Demo](docs/demo.gif)`
- Update Week 3 learnings section
- Review all sections for accuracy
- Make sure all badge links work

**Step 2: Clean up the repo**

- Remove any debug prints or commented-out code
- Make sure `.gitignore` covers everything
- Verify `.env.example` has all required variables
- Check that `requirements.txt` is complete

**Step 3: Write LinkedIn post**

```
Completed my Multilingual Content Moderator! 🛡️

3 weeks: PyTorch → FastAPI → Docker → Streamlit

What it does:
- Detects toxic content in English, Hindi, and Arabic
- 91.7% accuracy with language-aware threshold calibration
- REST API with batch processing (14.6 texts/sec)
- Interactive dashboard for live demos
- Dockerised — runs with one command
- 31 automated tests

Key insight: The same toxic message scores 0.93 in English
but only 0.42 in Hindi. Per-language thresholds fix this bias.

Built with: PyTorch, HuggingFace, FastAPI, Streamlit, Docker

GitHub: [link]

#MachineLearning #NLP #FastAPI #Docker #BuildInPublic
```

### Step 4: Final push

```powershell
git add .
git commit -m "Week 3 complete: Docker, Streamlit dashboard, portfolio ready"
git push
```

### Done when

- [ ] README is complete and professional
- [ ] Demo GIF in README
- [ ] LinkedIn post published
- [ ] Repo is clean — no debug code, no unnecessary files
- [ ] Project is portfolio-ready
- [ ] 🎉 PROJECT 1 COMPLETE!

---

## Week 3 Summary

| Day | Focus | Hours | Key Output |
|---|---|---|---|
| 1 | Docker fundamentals, fix Dockerfile | 2-3h | API running in container |

| Day | Focus | Hours | Key Output |
|---|---|---|---|
| 2 | Docker Compose, both services | 2-3h | API + Streamlit in containers |
| 3 | Streamlit: single text moderation | 2-3h | Interactive text analysis |
| 4 | Streamlit: batch upload + charts | 2-3h | CSV upload, visualisations |
| 5 | Streamlit: polish + history | 2-3h | Professional-looking dashboard |
| 6 | Integration testing + demo GIF | 2-3h | Everything working end-to-end |
| 7 | Final docs + LinkedIn | 1-2h | Portfolio-ready project |
| **Total** | | **14-19h** | **Complete, deployable project** |

## Key Concepts You'll Learn This Week

| Concept | What It Is | Where You'll Use It |
|---|---|---|
| Docker | Containerisation — package app + dependencies | Dockerfile |
| Multi-stage build | Keep images small by separating build and runtime | Dockerfile |
| docker-compose | Run multiple containers together | API + Streamlit |
| Volumes | Persist data between container restarts | Model cache |
| Streamlit | Python framework for interactive web apps | Dashboard |
| st.session_state | Persist data between Streamlit reruns | History table |
| Plotly | Interactive charts library | Category/language charts |
| st.file_uploader | Accept file uploads from users | CSV batch upload |

## Files You'll Create/Modify This Week

```
├── Dockerfile            ← Fix: Python 3.12, health check
├── .dockerignore         ← Create: skip venv, .git, __pycache__
├── docker-compose.yml    ← Update: environment variables
├── streamlit_app.py      ← Build: entire dashboard
├── README.md             ← Update: demo GIF, Week 3 learnings
```

```
└── docs/
    └── demo.gif          ← Create: screen recording of dashboard
```